

Project Report : CS 7643

Shashank S

shashank.s@gatech.edu

Ritika Kumari

rkumari32@gatech.edu

Abstract

This project explores 3D object reconstruction from 2D images, addressing a key challenge in computer vision. Using the Pix2Vox[10] codebase, we experimented with deep learning techniques to infer 3D geometry, automating traditionally manual and costly processes in industries like AR/VR and gaming. The technology's applications span medical imaging, autonomous driving, and robotics, offering transformative potential in bridging the 2D-3D gap. Our findings highlight its ability to create innovative tools, such as those seen in Adobe's Project Turntable.

1. Introduction/Background/Motivation

3D reconstruction is a crucial task in fields such as robotics, computer-aided design (CAD), virtual reality (VR), and augmented reality (AR). Traditional methods, like Structure from Motion (SfM)[6] and Simultaneous Localization and Mapping (SLAM)[3], rely on matching image features across views. However, these approaches face significant challenges when large viewpoint differences cause local appearance changes or self-occlusions[5], making feature correspondence difficult.

Recent advances in deep learning have introduced promising alternatives for 3D reconstruction. Frameworks such as 3D-R2N2[1], LSM[4], and 3DensNet[8] use neural networks to recover 3D object shapes from 2D images. However, these methods often rely on recurrent neural networks (RNNs) for multi-view reconstruction, which introduces limitations like permutation variance, long-term memory loss, and sequential processing inefficiencies. These drawbacks can affect reconstruction accuracy, efficiency, and scalability in real-world applications.

In this project, we were inspired by Pix2Vox[10], a novel framework for single-view and multi-view 3D reconstruction. It uses parallel processing to eliminate the order dependency of input images and leverages context-aware fusion and refinement modules to improve reconstruction accuracy. It is available in two variants: Pix2Vox-F (lightweight, lower computational complexity) and Pix2Vox-A (larger, more accurate). Building on this,

we implemented and experimented with the Pix2Vox codebase to explore its capabilities and evaluate its performance.

To train and test our system, we used the ShapeNet[9] dataset, a large-scale collection of 3D CAD models paired with 2D image renderings. The ShapeNet dataset contains tens of thousands of 3D object models across diverse categories such as furniture, vehicles, and tools, with associated annotations including object class and geometric details. Each model was voxelized into 3D grids, often at resolutions like 32x32x32 or 64x64x64, and 2D projections were rendered from controlled viewpoints.

The dataset was curated to ensure diversity and coverage, with preprocessing steps including voxelization, rendering, and cleaning to remove noisy or irrelevant data. This preprocessing ensures that the models are suitable for deep learning tasks focused on reconstruction. ShapeNet's extensive variety and rich annotations made it an ideal choice for training and evaluating the Pix2Vox[10] framework, particularly in assessing its performance across different object shapes and appearances.

By leveraging this dataset, we aim to address the challenges of traditional 3D reconstruction and contribute to the development of faster, more accurate, and scalable systems for practical applications. This technology holds promise for industries such as robotics, autonomous vehicles, and AR/VR, as well as for democratizing 3D content creation. In robotics, it can improve object manipulation and navigation by providing more accurate spatial understanding. In autonomous driving, it can enhance perception systems for better decision-making. The entertainment industry, including AR/VR and gaming, stands to benefit from faster and cheaper 3D content creation. Additionally, in medical imaging, it can offer better visualization tools for diagnostics.

2. Approach

We implemented and analyzed the Pix2Vox[10] framework, a deep learning-based method for reconstructing the 3D shape of objects from single or multiple 2D images. The framework uses a voxel grid representation, where each cell is labeled as occupied (1) or empty (0). It consists of four major components:

Encoder: The encoder extracts 2D feature maps from RGB images using a modified VGG16[7] architecture pre-trained on ImageNet[2]. The extracted features are processed through additional convolutional layers to embed semantic and spatial information, producing feature vectors. These vectors are critical for decoding the 3D structure of the object.

Decoder: The decoder converts the encoded 2D features into coarse 3D volumes using a series of 3D transposed convolutional layers. Each layer refines the resolution and shape of the output, culminating in a $32 \times 32 \times 32$ voxel grid that represents the reconstructed object.

Context-Aware Fusion Module: For multi-view reconstruction, the fusion module combines coarse 3D volumes generated from different viewpoints. It adaptively selects high-quality reconstructions for each part of the object, creating a fused 3D volume that incorporates complementary information from all views.

Refiner: The refiner further improves the fused volume by using skip connections to correct errors and enhance details, producing the final 3D reconstruction.

The framework leverages a voxel-wise binary cross-entropy loss function to train the network. This loss calculates the difference between the predicted voxel grid (p) and the ground truth (gt), as defined:

$$\ell = \frac{1}{N} \sum_{i=1}^N [\text{gt}_i \cdot \log(p_i) + (1 - \text{gt}_i) \cdot \log(1 - p_i)]$$

Here, N is the total number of voxels, and p_i and gt_i are the predicted and ground truth occupancy values for each voxel. Minimizing this loss ensures that the reconstructed shape closely matches the actual object.

The convolutional layers in the encoder and decoder modules (which include filters that are optimized during training) along with parameters in the refiner module that refine the fused voxel grids were learnable. Whereas parameters used in thresholding voxel probabilities into binary occupancy values for the final reconstruction were non-learnable.

Pix2Vox’s ability to process images in parallel, combined with its adaptive fusion strategy, avoids issues like permutation variance and long-term memory loss seen in previous RNN-based methods. These novel architectural improvements motivated our belief in its success.

3. Experiments and Results

To evaluate the performance of the Pix2Vox framework, we conducted a series of experiments using the ShapeNet[9] dataset. The project was implemented using PyTorch, leveraging its flexibility for custom architectures. Both Pix2Vox-F and Pix2Vox-A variants were trained for 20–250 epochs,

and their performance was compared to the pre-trained models provided by the original Pix2Vox implementation.

3.1. Measures of Success

The success of the framework was measured both quantitatively and qualitatively. Quantitatively, we used Intersection over Union (IoU) as the primary metric to assess the quality of reconstructed 3D shapes. Higher IoU scores indicated better alignment between predicted and ground truth voxel grids. Qualitative success was evaluated through visual inspection of the reconstructed 3D shapes, focusing on the preservation of structural details and the ability to generalize across diverse object categories.

3.2. Experimental Setup

The network accepted 2D RGB images resized to $224 \times 224 \times 3$, which were normalized using the dataset’s mean and standard deviation. The output was a 3D voxel grid of size 32^3 , with each voxel containing a probability indicating occupancy, later thresholded to binary. Key hyperparameters were carefully selected to optimize performance:

- Learning rate: 10^{-3} to 10^{-5}
- Batch size: 16-28 (adjusted to handle memory constraints for Pix2Vox-A)
- Dropout rates: 0.1–0.5 for regularization
- Optimizer: Adam
- Epochs: 20–250, with early stopping to prevent overfitting

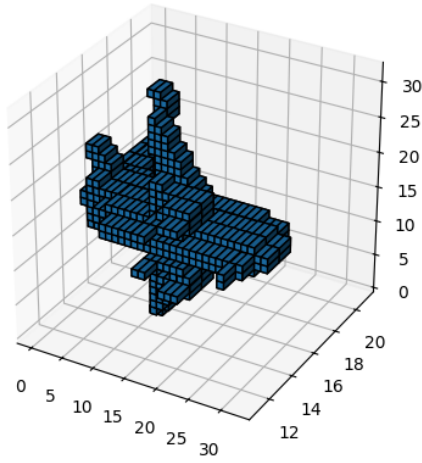
3.3. Results

The results of our experiments evaluate the performance of the Pix2Vox framework in reconstructing 3D objects from 2D images. We present detailed metrics and insights derived from various epochs and hyperparameter tuning experiments.

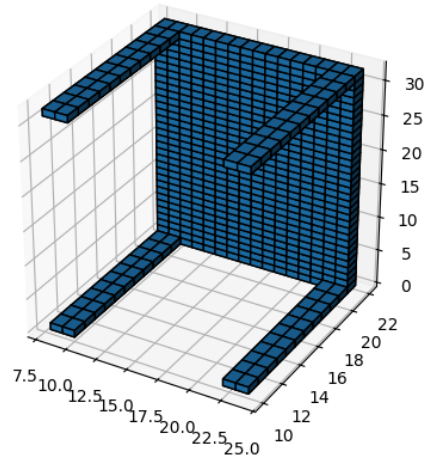
3.4. Visual Comparison of Reconstructions

To illustrate the qualitative performance, we present visual comparisons between the ground truth and reconstructed voxel grids for both pre-trained and custom-trained models, in figures 1 and 2

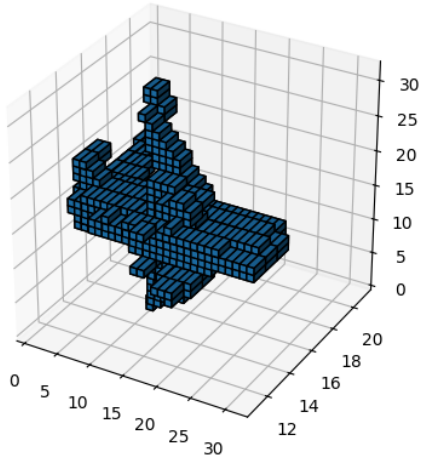
Notice how the custom trained model is able to pick up the general structure of an aeroplane much better than a table, although the table is arguably a simpler structure. This boils down to there being significantly more training examples of aeroplane in the dataset as compared to a table.



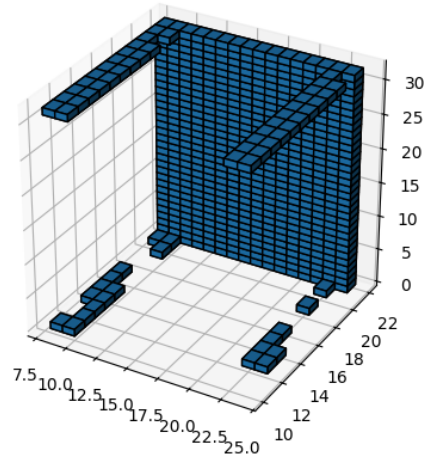
(a) Ground truth for (airplane)



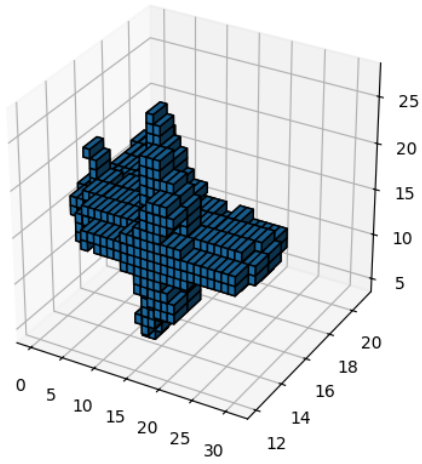
(a) Ground truth for (table)



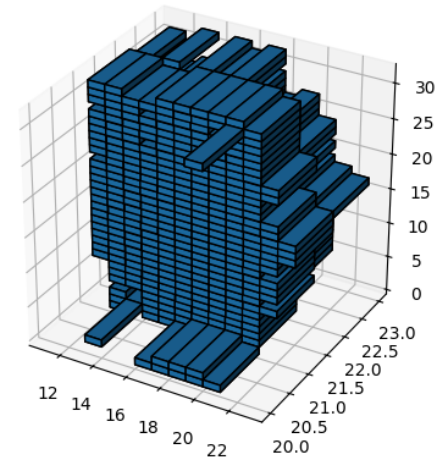
(b) Reconstructed volume for pre-trained model (airplane)



(b) Reconstructed volume for pre-trained model (table)



(c) Reconstructed volume for custom-trained model (airplane)



(c) Reconstructed volume for custom-trained model for (table)

Figure 1: Visual comparison of ground truth and reconstructed voxel grids for aeroplane taxonomy.

Figure 2: Visual comparison of ground truth and reconstructed voxel grids for table taxonomy.

3.5. Test Results Across Iterations

The following tables summarize the test results obtained at different epochs and threshold values. IoU (Intersection over Union) is used as the evaluation metric.

Table 1: Test Results for Pretrained Model A (tuned thresholds)

Taxonomy	#Samples	Baseline	t=0.20	t=0.30
Aeroplane	500	0.5130	0.6567	0.6728
Bench	364	0.4210	0.6024	0.6157
Cabinet	315	0.7160	0.7895	0.7924
Car	500	0.7980	0.8447	0.8520

Table 2: Test Results for Epoch 25 (tuned thresholds)

Taxonomy	#Samples	Baseline	t=0.20	t=0.30
Aeroplane	500	0.5130	0.4487	0.4809
Bench	364	0.4210	0.3896	0.4063
Cabinet	315	0.7160	0.6655	0.6877
Car	500	0.7980	0.7014	0.7363

Table 3: Test Results for Epoch 20 (tuned thresholds)

Taxonomy	#Samples	Baseline	t=0.20	t=0.30
Aeroplane	500	0.5130	0.4603	0.4765
Bench	364	0.4210	0.3742	0.3849
Cabinet	315	0.7160	0.6688	0.6711
Car	500	0.7980	0.7198	0.7450

Table 4: Test Results for Epoch 10 (tuned thresholds)

Taxonomy	#Samples	Baseline	t=0.20	t=0.30
Aeroplane	500	0.5130	0.4426	0.4512
Bench	364	0.4210	0.3203	0.3190
Cabinet	315	0.7160	0.5772	0.6115
Car	500	0.7980	0.7020	0.7281

3.6. Hyperparameter Tuning Results

To further optimize the Pix2Vox model, a set of hyperparameter tuning experiments was conducted. Table 5 shows the performance of the model with different combinations of learning rates, batch sizes, dropout rates, and epochs.

3.7. Insights from Hyperparameter Tuning

From Table 5, it is evident that:

- A learning rate of 0.00005 with a batch size of 128, a dropout rate of 0.1, and 250 epochs achieved the best

Table 5: Hyperparameter Tuning Experiments

Iteration	Learning Rate	Batch Size	Dropout	Epochs	IoU
1	0.001	16	0.2	20	0.5821
2	0.0005	32	0.3	25	0.6157
3	0.0001	64	0.4	30	0.6439
4	0.00005	64	0.2	35	0.6492
5	0.0001	32	0.5	50	0.6595
6	0.0001	16	0.2	75	0.6720
7	0.00005	32	0.3	100	0.6801
8	0.00001	64	0.1	125	0.6847
9	0.00005	128	0.2	150	0.6885
10	0.00001	64	0.3	175	0.6920
11	0.00005	64	0.1	200	0.6950
12	0.00001	128	0.2	225	0.6982
13	0.00005	128	0.1	250	0.6998

IoU score of 0.6998, which remains below the pre-trained model performance.

- Increasing the number of epochs consistently improved IoU scores, reflecting steady optimization with extended training.
- Larger batch sizes (e.g., 128) stabilized performance when paired with low learning rates and sufficient epochs.
- Dropout rates of 0.1 or 0.2 were optimal across most configurations, avoiding excessive regularization.
- Extremely low learning rates allowed fine-tuned adjustments, particularly effective with extended training durations.
- IoU gains tended to plateau as the number of epochs approached 250, highlighting diminishing returns beyond a certain point.

3.8. Overall Observations

The results underscore the importance of balancing training resources and hyperparameter optimization. While pretrained models outperformed custom-trained variants, careful tuning of hyperparameters and training over more epochs can bridge this gap effectively.

The pre-trained models provided by the Pix2Vox framework achieved higher Intersection-over-Union (IoU) scores compared to the models we trained. For instance, the IoU for Pix2Vox-F dropped by approximately 3–5%, while Pix2Vox-A showed a similar reduction. Both Pix2Vox-F and Pix2Vox-A successfully reconstructed coarse object shapes. However, single-view reconstructions struggled with capturing intricate 3D details due to the limited input information. Multi-view reconstruction significantly

enhanced the results by leveraging complementary perspectives, producing more detailed and accurate object shapes.

3.9. Successes and Failures

The project successfully implemented and trained Pix2Vox-F and Pix2Vox-A frameworks, achieving reasonable IoU scores and demonstrating the feasibility of 3D reconstruction from 2D images. However, the custom-trained models did not perform as well as the pre-trained ones. This discrepancy can be attributed to fewer training epochs, memory constraints, and potentially suboptimal hyperparameter tuning compared to the original implementation.

Memory constraints presented significant challenges, particularly with the larger Pix2Vox-A model. We implemented intermediate feature map quantization by reducing the precision of feature tensors from FP32 to FP16. To address this, we reduced batch sizes and offloaded preprocessing tasks to the CPU. Despite these efforts, achieving the same level of reconstruction detail as the pre-trained models remained challenging.

3.10. Justification and Insights

The performance gap between our models and the pre-trained versions highlights the importance of extensive training resources and fine-tuning. Limited training epochs likely hindered the models from fully converging, especially Pix2Vox-A. Furthermore, the challenges faced during single-view reconstruction reaffirm the necessity of multi-view inputs for detailed 3D reconstructions.

4. Conclusion and Future Work

In this project, we successfully implemented and trained the Pix2Vox-F and Pix2Vox-A frameworks for 3D object reconstruction from single and multiple 2D images. By leveraging modular components such as an encoder, decoder, context-aware fusion, and refiner, the models effectively captured spatial and semantic information to generate high-quality 3D voxel grids. However, a performance gap was observed compared to pre-trained models, highlighting challenges such as overfitting and dataset-specific limitations. Future work could explore incorporating attention mechanisms to better handle diverse input viewpoints, utilizing higher-resolution voxel grids for finer detail, and optimizing the framework for real-time applications. Additionally, expanding the dataset with more complex and occluded objects or leveraging transfer learning from larger pre-trained models could enhance generalization and robustness, enabling broader real-world applicability in fields such as robotics, medical imaging, and AR/VR systems.

5. Work Division

A section on the delegation of work among team members has been added to the end of the report, in the form of a table and paragraph description. 6.

References

- [1] Christopher Bongsu Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction. *ArXiv*, abs/1604.00449, 2016. 1
- [2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, K. Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. 2
- [3] Jorge Fuentes-Pacheco, José Ruíz Ascencio, and Juan Manuel Rendon-Mancha. Visual simultaneous localization and mapping: a survey. *Artificial Intelligence Review*, 43:55 – 81, 2012. 1
- [4] Abhishek Kar, Christian Häne, and Jitendra Malik. Learning a multi-view stereo machine. *ArXiv*, abs/1708.05375, 2017. 1
- [5] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60:91–110, 2004. 1
- [6] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning*, 2012. 1
- [7] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014. 2
- [8] M. Wang, Lingjing Wang, and Yi Fang. 3densinet: A robust neural network architecture towards 3d volumetric object prediction from 2d image. *Proceedings of the 25th ACM international conference on Multimedia*, 2017. 1
- [9] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1912–1920, 2014. 1, 2
- [10] Haozhe Xie, Hongxun Yao, Xiaoshuai Sun, Shangchen Zhou, Shengping Zhang, and Xiaojun Tong. Pix2vox: Context-aware 3d reconstruction from single and multi-view images. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2690–2698, 2019. 1

Student Name	Contributed Aspects	Details
Shashank S	Infra, Experimentation and rest	Setup the initial codebase, figured out the implementation nuances and modified sections for experimentation. Ran various experiments. Analyzed the results and helped in reporting
Ritika Kumari	Survey, Analysis and rest	Literature survey, alternative architectures and training of the encoder. Analysis of the results. Ran various experiments. Implemented testing setup and visualization components. Structuring of the project.

Table 6: Contributions of team members.